Abstract

1 Introduction

Stream Processing (SP) used a lot in the C2EC, systems like Smart Grids and Vehicular Networks to name a few.
One of the reasons is popular is scalable stateful analysis like streaming aggregation, which allows to continuously aggregate time-windows of raw data to e.g., detect patterns or create informative summaries.
Important to note here is that SP frameworks, SPEs, have been mostly designed for data centers hardware, and their use towards the edge end of the C2EC is challenged by the limited available resources. Note what matters in such devices what is their nominal computational power, but rather the one they can devote to a certain application. A modern vehicle could for instance mount a powerful device [] but still limited resources for a monitoring application to run on it.
In this case, techniques that allow trade-offs certain resources over other, like the one in focus in this paper, memory compression for stateful aggregation, can be beneficial to fit applications to the devices found in some specific portions of the C2EC.
challenge: As preliminary discussed in [1], the costs of benefits of memory compression for stateful streaming aggregation depend on the specifics of the aggregation at hand and the data such aggregation is applied to. Since the latter evolves over time, so does the best configuration of memory compression to meet a given trade-offs balancing. In this context, Reinforcement Learning (RL) is a promising technique to automate the decision-making process, as also done in related SP contexts such as JINGYU. The novel and previously unexplored topic of RL-based compression in streaming aggregation, though, raises new challenges. More concretely:

Contribution:

Organization: ...

2 Preliminaries

Stream Processing and streaming aggregation

Reinforcement Learning ← JINGYU

REUSE FROM PREVIOUS PAPER
Stream, tuples, event-time, timestamp, schema.
Aggregate, general definition and pseudo-code.
The pseudo-code can also have the compression parts, with a different color
Cover the streaming aggregation part, explain the D parameter.
Then say that since the paper is about RL + compression and D is the parameter of interest for the RL, namely changing it over time. Important to notice a decrease in D has immediate effect but a decrease takes time } probably this part can go in §6. Maybe show something of this in Eval too?

3 Problem definition

Our goal is to shape, both from a theoretical as well as a practical perspective, an approach to leverage RL for streaming aggregation.
To be succesfull, such an approach needs to align the progress of the RL agent (which happens in discrete action, reward steps (see §2.2)) and that of the aggregate (which evolves continuosly based both on wall-clock and event-time (see §2.1)), as we discuss in §4.

Moreover, it also need to define a learning environment in which episodes can be started efficiently, easing the cost of deployment and state initialization for streaming aggregates, as discussed in §5.
Least but not last, it also need to define a RL specification in terms of state, action, reward able to effectively evolve an agent despite the time-consuming steps within an episode, as discussed in §6.
To assess the quality of an agent and its ability to choose the best compression threshold (maybe good to connect before to the D parameter and be explicit about this), we compare it with several baselines, each keeping a fixed D value, and one choosing random D values, along common performance metrics such as
: throughput
: latency
: compression ratio
: CPU consumption
: ...
} we need to define these. Need to decide whether to put all the ones part of the state here or not... probably not since they are not used to assess quality.

→ The rationale is that constant-D baselines will choose a safe, yet too-light compression, an unsafe and aggressive one, or some compression level whose quality depends on the underlying fluctuations of data (no matter the underlying data volumes and distribution characteristics), while the RL agent will be able to continuously choose the best D value. The random D baseline will then be used to prove picking the random D value is not a trivial process in which all actions have similar effects.

4 "Time alignment"

5 "RL-environment"

6 RL model

7 Evaluation

[overview of what is contained in this section]

Software/Hardware setup

Use cases
- Linear Road
- Synthetic

Baseline study
In order to assess the learning quality of our RL agent, we first study how baselines perform with respect to the compression, latency, and CPU metrics for various constant D values and D values chosen randomly. For constant D values, we aim at dividing possible D values into three ranges:
- safe: leading to no latency violation no matter the point in time such D value is chosen
- worth: leading to no latency violations only if selected at the right time
- unsafe: leading to a latency violation if selected in more than 90% of the cases.

Say random is to show the RL agent instead needs to learn
A good RL agent will learn not to enter unsafe and stay closer to worth than to safe
- explain that what we want is an agent that compresses as much as possible without exceeding a threshold latency (maybe in problem definition?)

Precondition:
- Show how rate, latency, CPU behave when no compression
- Say D1.0 ... D0.9, D^* refer to the baselines

Postcondition:
- explain we run X episodes each of Y steps, that we compute the average value of each metric and then further aggregate them into box-plots. Say how long each episode takes

A possible question here could be: why not running the entire query with that compression?

Need to fix script, random should be on 3 actions, not 11!

Need to say the X episodes start at random points within this period and that this period is chosen so that we can fill the state (left side) and make sure the episode has data (right side).

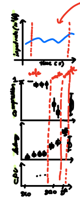

Figure X shows how the input rate evolves over the X hours covered by the data. As shown, EXPLAIN

Figure x contains, for each baseline, the boxplot of the averages from each episode of the compression, latency, and CPU metrics.
As shown, D values in between X-Y are safe, EXPLAIN
" " " " " " " worth, EXPLAIN
" " " " " " " unsafe, "
Also, not as easy as random, since random actions mean violations →

As shown, by selecting D values following the random strategy, it is possible to achieve high compression. This is because high compression can be achieved for $D \in [X, Y]$ and, based on how random values are chosen by the D^* baseline, the probability of not picking any such value during the X steps of each episode is x, and also because, as shown in Figure x, once a certain compression level has been reached, an immediate increase in D does not imply a decrease in the compression ratio.

speculation for now, we see what happens when we have the data.

[THIS IS FOR LINEAR ROAD, THEN DO THE SAME FOR SYNTHETIC]

8 Related Work

9 Conclusions/Future work

- More parameters (e.g., type of compression)
- different compression algorithms, and maybe many of them co-existing and becoming a parameter too
- exploring RL on top of streaming aggregation to control trade-offs, for instance by means of approximation
- speed-up learning by means of simulation (maybe surrogate is a term that can be used in RL too?)